Kubernetes Notes

Personal Reference

Last updated: July 23, 2026

Contents

---

1 Introduction

The purpose of this document is, obviously, my Kubernetes notes for myself. I am trying to avoid buzzwords and jargon, and explain both the *hows* and *whys* of Kubernetes.

That implies two basic questions worth answering before we go any further:

- What *is* Kubernetes?
- Why does Kubernetes exist – and why have so many people and organizations found it useful?

You'll often see Kubernetes described as "the operating system of the cloud." This is a buzzy way of describing it, but it is an accurate description. The different components of a Kubernetes cluster facilitate deploying applications in a cloud environment. That cloud may be public, like Amazon Web Services, or private, like your employer or school's server racks.

Software suited for running on a cloud should be durable and largely agnostic to what computer it runs on, aside from fundamental incompatibilities on the level of x86 vs. ARM processor architectures, Linux vs. Windows vs. macOS, and so on. As operating systems provide convenient interfaces to the physical computer programs run on, they (ideally!) remove a programmer's need to know the specific CPU, RAM, and networking details of the machine their code runs on. Kubernetes does the same one level up – abstracting away individual servers, nodes, regions, and storage types, handling concerns like application self-healing, upgrades and rollbacks, and storage on the programmer's behalf. This may not be a perfect analogy, but none is. It is the springboard for understanding those *hows* and *whys*.

Kubernetes was originally developed at Google, to make it easier for programmers to deploy and maintain programs at so-called "Google scale;" highly available, highly fault-tolerant systems. It is now owned and maintained by the Cloud Native Computing Foundation (CNCF). Before Kubernetes' release, containerization tools – Docker being the most known – were gaining notice and popularity for deploying industry applications. Docker handles a lot of the repeated complexities of deploying applications; packaging up everything you need to run the application, with a smaller footprint than a VM. However, Docker and Podman are *containerization* tools. Their purpose is to set up a lightweight-but-isolated environment to run an application in. Things like application auto-healing and rollouts are not in scope for those programs. While Docker provides related tools for some of these, specifically Docker Compose and Docker Swarm, a solution more suitable for production deployments was still needed. Enter Kubernetes.

2 Components of a Kubernetes Cluster

A Kubernetes cluster consists of a **control plane node** (the "leader," "brain," or "main office" of the cluster) and one or more **worker nodes**. The control plane manages the desired state of the cluster; worker nodes run the actual workloads. Certain deployments, typically home lab or other sorts of non-production Kubernetes clusters, may run a single node which hosts both the control plane's components and runs workloads, but this is not advisable for production.

2.1 API Server

Coming soon!

2.2 `etcd`

A core component of Kubernetes is the durable `etcd` database. `etcd` is a key-value database (as opposed to a relational/SQL database, or a document store like MongoDB) used by many other distributed systems. `etcd` is a Kubernetes cluster's "source of truth" for the *desired cluster state*; a concept that is crucial to understanding and using Kubernetes.

Like most well-engineered pieces of software, access to the database is not a free-for-all affair. The Kubernetes API server is the intermediary between the `etcd` database and the outside

world. The rest of this subsection discusses `etcd` and how Kubernetes uses it in greater depth; readers for whom this is not interesting or relevant can skip it.

High-availability Kubernetes servers may want to run multiple `etcd` instances; I will go into more detail on this later. Clusters using multiple `etcd`s use the *raft consensus algorithm* to designate a "leader" or main instance.

`etcd` consists of three executables:

1. `etcd` - the program running the database itself.
2. `etcdctl` - provides a command-line interface for querying the database and handling admin tasks
3. `etcdutl` - used for tasks that require operating directly on `etcd` data files, like migrating data between versions of `etcd`, restoring snapshots, defragmenting the database, and database validation

2.2.1 How Kubernetes stores data in `etcd`

`etcd` is a key-value database. Possible values for keys are seemingly pretty free-form, but Kubernetes uses a Linux path-like format for them. All keys start with `/registry/`, followed by the resource type, the namespace (if the object is namespaced), then the resource's name.

For example, the default namespace's definition is at `/registry/namespaces/default/`, and namespaced objects like Pods will live at, e.g. `/registry/pods/myAppNS/apiPod/`.

2.2.2 Using `etcdctl`

In most setups, `etcdctl` will require you to specify the API version, the endpoint and port the database server is listening at, and the certificate info. For a concrete example, if you're running a cluster locally using Docker Desktop, from within the Pod running `etcd`, your `etcdctl` commands may look something like

```
ETCDCTL_API=3 etcdctl get /registry/namespaces/default \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

The values for the Kubernetes objects are stored in Protobuf format, so while this command should run successfully (at least on a cluster created by Docker Desktop), the output will not be human-readable. There are multiple ways to turn the Protobuf data into more readable formats, but the primary one is, of course, the Kubernetes API.

To view the values in a human-readable form without leaving `etcdctl`, pass the `-w` flag (for "write," as in, write out to your terminal/display). Options include `fields`, `simple`, and `json`. `fields` and `json` will render the values for individual keys in a human-readable format, in my experience.

2.2.3 Commands and Flags

Here are some `etcdctl` commands and flags worth knowing:

- `etcdctl get|put|del` - see, set, or delete entries.
- `etcdctl get --prefix` - get all items whose key starts with the given prefix.

- `etcdctl get --keys-only` - get only the key, not the value, if it exists. Useful in conjunction with `--prefix` to get every object of a specific type, in a specific namespace, or both.

Further reading

- https://etcd.io/docs/v3.6/

2.3 Controller Manager

The *controller manager* is a daemon that manages controllers. This implies a very obvious question – what is a controller? Controllers programs that monitor the cluster (through the API server, Kubelet, or both?) to it to reconcile the desired and actual state of your cluster. Each controller manages a type or type of objects. For example, the *Node manager* checks each Node's health every five seconds. If a Node goes down, with a grace period of forty seconds, the Node manager recreates its' Pods on another node.

Additional controllers for Custom Resource Definitions (CRDs) can be added.

2.4 Scheduler

The *scheduler* is the service that exposes

2.5 Container Runtime

Every Node, worker or Control Plane, must have at least one *container runtime* installed on it. Container runtimes, such as Docker Engine, handle the crucial job of actually running containers. While Docker Engine is the most well-known, it can be heavyweight for some deployments. Other runtimes include containerd, CRI-O, and Mirantis Container Runtime, which is a commercially available runtime.

All container runtimes should be compatible with the Container Runtime Interface (CRI), which is an interface or protocol for a uniform method of communication between the kubelet and container runtime.

CLI applications to interact with runtimes include `ctr`, `nerdctl`, and `crictl`. `crictl` is used for debugging purposes, `nerdctl` is a general-purpose Docker-like CLI application specifically for containerd, and `crictl` is another debugging-focused tool for all CRI-compatible runtimes, associated with Kubernetes.

Each of these applications connects to Unix sockets suck as:

- `unix:///var/run/dockershim.sock`
- `unix:///var/run/containerd/containerd.sock`
- `unix:///var/run/crio/crio.sock`
- `unix:///var/run/cri-dockerd.sock`

2.5.1 Common Container Runtime Commands

Coming soon!

2.6 Kubelet

The kubelet is an application that runs on each Node, with three major responsibilities:

1. registering the Node with the Control Plane upon Node start-up
2. managing communication between the Node and the Control Plane
3. managing Pods so that the container runtime can run their constituent containers, by doing things like:
   (a) mounting Volumes, ConfigMaps, Secrets
   (b) running liveness and readiness probes, to monitor Pod health
   (c) checking they respect any resource limits defined for them

The kubelet communicates with the Control Plane primarily through the API Server, watching for any updates to PodSpecs (the specification component of a Pod manifest) scheduled to its specific Node. It also posts updates back to the API Server about the *actual state* of those *desired resources*.

The kubelet also runs its own small HTTPS API, through which it proxies requests that operate on the container-level, like `kubectl logs` and `kubectl exec`.

Aside from changes to the cluster state requested through `kubectl`, which are assigned to a Node by the scheduler and communicated to the kubelet via the API Server, the kubelet can also manage *static Pods*. More on those later.

The kubelet is typically run as a daemon with `systemctl`, since it cannot manage itself as a Pod.

2.7 Kube-Proxy

Coming soon!

3 Objects in Kubernetes

3.1 Pods

The Pod is the minimal "unit of work" in Kubernetes. Pods are a wrapper around containers, managing the basics of provisioning and running containers. This includes things like storage and networking.

3.1.1 Multi-Container Pods

3.1.2 Static Pods

Static Pods are created directly on a desired Node by putting their manifest YAMLs in a dedicated path on the Node–so without `kubectl create`, `apply`, or `run`. The host Node's Kubelet will always restart a static Pod if it crashes, giving them an enhanced level of predictability and stability.

The standard path for static Pod manifests is `/etc/kubernetes/manifests/`; however, this can be set to whatever you like using either the `-pod-manifest-path` flag passed to the Kubelet, or in the Kubelet's config file as `staticPodPath`. The standard path for the Kubelet config file is `/var/lib/kubelet/config.yaml`, but you can always `ps aux | grep kubelet` for it.

The Kubelet watches both the static Pod path and the API server. If a Pod is created via putting a manfiest in the static Pod path, the static Pod is reported to the API server, has its Node's name appended to the name set in the manifest, and cannot be deleted through things like `kubectl` or the Kubernetes HTTP API.

One use case for static Pods is to create Pod manifests for each Control Plane component and storing them in a Node's static Pod path. This is how `kubeadm` sets up and manages clusters, in fact!

It is also worth noting that both static Pods and DaemonSets are ignored by the scheduler. For DaemonSets, each Node has an instance of the Pod. DaemonSets are elucidated upon in their dedicated section.

3.1.3 Manifests

3.2 Namespaces

The Namespace is the simplest way to manage resource and workflow isolation in a Kubernetes cluster. All Kubernetes object types are either "namespaced" or not; this determines whether they are visible across the entire cluster or only to objects in their Namespace. Objects like ConfigMaps, Pods (and anything that has Pods, like Deployments, Jobs, and CronJobs), and Secrets are namespaced. Meanwhile Nodes and Volumes are not namespaced. A solid rule of thumb is that objects that correspond closely to to either physical hardware or cluster-wide information are probably not namespaced; however you can always check whether an object type is namespaced by running `kubectl api-resources` to get a list of all object types available to the cluster of your current context.

On cluster startup, Kubernetes creates three Namespaces:

- `default`
- `kube-system`
- `kube-public`

3.3 Deployments

... ReplicaSets and ReplicationControllers

3.4 DaemonSets

3.5 Services

4 Storage

Kubernetes, as with all container tools, works with a stateless paradigm. Pods (and therefore implicitly containers) should be edited sparingly, because the purpose of Kubernetes is to deploy, scale, and self-heal applications without need tor additional setup beyond the cluster existing. But state is needed. There are many obvious uses for state–mutating the state of the file system with logs, the contents of a database, etc.–and using Kubernetes does not remove the need to manage that state.

Kubernetes has tools for handling state. This section discusses how containers handle state, how to define, manage, and persist state for applications deployed in Kubernetes, and how to integrate with cloud storage providers like Amazon's S3.

4.1 Images, Containers and State

Each line or command in a Dockerfile adds a new *layer* to the image. Each layer is a specific change to the filesystem defined by the layer created by the last line.

For an example from one of my own Dockerfiles:

```
COPY pyproject.toml poetry.lock /app
RUN pip install --upgrade pip && \
    pip install poetry && \
    poetry config virtualenvs.create false && \
    poetry install --no-root

COPY . /app
```

The `COPY` commands define a new layer with the files from the host filesystem in the desired directory of the image's filesystem. The `RUN` commands execute their arguments and create a new layer storing the changes those commands made–in this example, those being to the Python installation in the image's filesystem, including changes to the system Python in the image, the `site-packages` directory which stores third-party libraries installed by `pip`, and so on.

5 `Kubectl`

6 Cookbook

This section contains tips and guides on how to handle common tasks related to maintaining Kubernetes clusters or managing workloads deployed on them.

6.1 Upgrading a Cluster

These instructions are primarily for Debian-based distros using `apt`.

6.1.1 Clusters Provisioned with `kubeadm`

We can break upgrading a `kubeadm` cluster into two parts: upgrading the Control Plane node(s) and upgrading the workers.

Control Plane

You first need to check which package repository your OS is using; there is a separate repository for every minor version of Kubernetes. Ensure it's set to the minor release of Kubernetes you want to use.

On Debian and Ubuntu, this is stored at `/etc/apt/sources.list.d/kubernetes.list`.

On RHEL, Fedora, and CentOS, it should be located at `/etc/yum.repos.d/kubernetes.repo`.

Regardless of your distro, in the file, the package repo URL should match a pattern similar to:

```
https://pkgs.k8s.io/core:/stable:/v1.36
```

Change the version to the minor version you're upgrading to.

Once the package repo is updated, instruct the package manager to update its package database with an `apt update` and then identify the specific patch version you wish to upgrade to with `apt-cache madison kubeadm`. `apt-cache madison` searches the local `apt` cache for packages matching the and prints info about them to stdout. Following is the output I got from `sudo apt-cache madison kubeadm`:

```
kubeadm | 1.36.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb
   Packages
kubeadm | 1.36.2-2.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb
   Packages
```

```
kubeadm | 1.36.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb
   Packages
kubeadm | 1.36.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.36/deb
   Packages
```

Temporarily unhold the Control Plane's `kubeadm`, update it to your desired new version, and verify:

```
sudo apt-mark unhold kubeadm && \
sudo apt-get update && \
sudo apt-get install -y kubeadm='1.36.3-1.1' && \
sudo apt-mark hold kubeadm
kubeadm version
```

With your Control Plane's new version of `kubeadm` installed, plan the upgrade with `sudo kubeadm upgrade plan`. If all is well, run the upgrade: `sudo kubeadm upgrade apply v1.36.3`. The success message will look like this:

```
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.36.3".
   Enjoy!

[upgrade/kubelet] Now that your control plane is upgraded, please
   proceed with upgrading your kubelets if you haven't already done so
   .
```

If you're running a high availability setup with multiple Control Plane nodes, you'll need to repeat the preceding with a few slight differences. Update the apt repository, but instead of `kubeadm upgrade apply`, run `kubeadm upgrade node`. Additionally, you don't need to upgrade the CNI plugin again, since all CRDs, Deployments, etc., across the cluster are upgraded in one shot.

Before upgrading the Kubelet and `kubectl`, cordon the Node and drain it. Then temporarily unhold the packages for them to update to the same minor version you upgraded `kubeadm` to. Finally, use `systemctl` to restart the Kubelet:

```
# Update the packages:
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && \
sudo apt-get install -y kubelet='1.36.3-1.1'  kubectl='1.36.3-1.1' &&
   \
sudo apt-mark hold kubelet kubectl
# Restart the service:
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

After all this, you can uncordon the node.

Workers

Each worker node should be cordoned and drained to avoid disrupting anything running on it.

Once everything running on the Node has been moved, begin the upgrade process:

1. Upgrade the Node's `kubeadm`.
2. Run `sudo kubeadm upgrade node` to upgrade its' Kubelet's config,.
3. Upgrade `kubectl` and the Kubelet.

4. Restart the Kubelet service with `systemctl`).
5. Uncordon the Node.

For more detailed explanations for any of these steps, reference the Control Plane section.

6.1.2 Upgrading `etcd`

6.2 Backing up and restoring `etcd`

6.2.1 Backing up `etcd`

6.2.2 Restoring an `etcd` backup

6.3 Dealing with IP address exhaustion